Session 23 Homework — Issue Detail: a third real nested route, then a real bug you find yourself

CORE, STRETCH, and CHALLENGE are all required — every one of them must be completed. The three names describe increasing difficulty only, nothing about which parts you submit.

Context (required)

By the end of class, Project Detail's own real issue list already links to `/workspaces/[id]/projects/[projectId]/issues/[issueId]` — a real URL nothing renders yet. This homework builds that real page, then finds and fixes a real bug hiding in a pattern you have already typed three times.

Goal (required)

A real Issue Detail page, following the identical real skeleton/fetch-guard/render pattern LIVE and TEAMWORK both already used. Then: a real, disclosed bug in that exact pattern, reproduced on purpose, then fixed on all three real pages (CORE+STRETCH). A real, self-researched written answer about static vs. dynamic rendering (CHALLENGE).

What you already have

A real `apiFetch` helper; real `Workspace` / `Project` types; two real working dynamic routes (`/workspaces/[id]` , `/workspaces/[id]/projects/[projectId]`), each with its own real `loading` / `not-found` / `error` trio; a real backend where `GET /issues/:id` is genuinely public.

What to build (required)

A real third nested route, `/workspaces/[id]/projects/[projectId]/issues/[issueId]` , rendering one real issue.

CORE (required)

How to start

Build the real skeleton first (an `async` function, `params` as a real `Promise` with THREE real fields this time, echoing `issueId` back) — prove the real route resolves before fetching anything, the identical real discipline LIVE and TEAMWORK both already followed.

Hints

- **Hint 1 (conceptual):** `assigneeId` / `reporterId` on the real `Issue` type are raw real user ids, not names. Resolving a real name needs `GET /users/:id` — check whether that real route is public or protected before deciding what this page can honestly show today.
- **Hint 2 (where to look):** `api/src/users/users.controller.ts` in the real backend repo.

- **Hint 3 (where to look):** the identical real `loading.tsx` / `not-found.tsx` / `error.tsx` trio you already have twice — write the third application yourself; do not wait for the walkthrough.

STRETCH (required)

How to start

Run this for real, against your own real running backend, before changing any code:

BASH

```
curl -s http://localhost:3000/issues/abc
```

Read the real response. Then visit `/workspaces/1/projects/1/issues/abc` in your own real browser and look closely at what actually renders.

Hints

- **Hint 1 (conceptual):** every CORE page this session checks `res.status === 404`. Is that the ONLY real non-2xx status code a real backend can ever return for a malformed id?
- **Hint 2 (where to look):** `ParseIntPipe`, already used on every real `:id` route parameter in the backend you did not write this session — check what it actually does with a non-numeric value.
- **Hint 3 (API name):** `Response.ok` (the real Fetch API property) is true for every real 2xx status, and only those.

CHALLENGE (required)

How to start

Run `npm run build` inside `web/` and read the real output table — every real route this session built is marked `ƒ` (server-rendered on demand), never `○` (prerendered as static content).

Hints

- **Hint 1 (conceptual):** Next.js's own real `generateStaticParams` function can pre-render a real dynamic route's own known ids at build time.
- **Hint 2 (where to look):** this exact installed Next.js version's own real documentation (`node_modules/next/dist/docs/`) for `generateStaticParams` and the static/dynamic rendering model — not an older tutorial's own description, which may genuinely no longer apply.

What to research

1. What real build-time inputs would `generateStaticParams` need to pre-render every real issue id for this exact page?
2. Give one concrete, real reason pre-rendering every real issue at build time would be a genuinely bad fit for this app.
3. Your own real, justified recommendation: should this page stay dynamic, or could a narrower real subset be safely pre-rendered?

Rules (required)

Allowed: everything already shown LIVE/TEAMWORK this session, plus the real official Next.js documentation for `generateStaticParams` and the Fetch API.

Not yet: no real name resolution for `assigneeId` / `reporterId` — `GET /users/:id` is genuinely protected; that arrives for real in Session 25. No Tailwind, no client-side state, no auth.

Test it (design your own full proof — this is a starting point, not the complete list)

inside api/

BASH

```
PORT=3000 npm run start
```

inside web/ (second terminal)

BASH

```
npm run dev
```

in a third terminal

BASH

```
curl -s http://localhost:3001/workspaces/1/projects/1/issues/1
curl -s http://localhost:3001/workspaces/1/projects/1/issues/999999
curl -s http://localhost:3001/workspaces/1/projects/1/issues/abc
curl -s http://localhost:3001/workspaces/abc
curl -s http://localhost:3001/workspaces/1/projects/abc
npx tsc --noEmit
```

Common confusions

- Checking `res.status === 400` as a SECOND, separate condition instead of real `!res.ok` — works for this one real case, but `!res.ok` is the real, direct property built for every real non-2xx status at once.
- Fixing the real bug on only the one page where you first noticed it — all three real pages share the identical real mistake.
- Inventing a fake assignee/reporter name instead of showing the real raw id.

Done when (required)

- Issue Detail renders a real issue's own real fields; a real missing issue id shows your own real not-found text.
- A real non-numeric id on ANY of the three real dynamic routes shows the real not-found text, not "undefined".

- `npx tsc --noEmit` exits clean.
- Your written CHALLENGE answer covers all three required points above.

Working time (required)

About 50 minutes total (HW-01..11) — CORE (Issue Detail, HW-01..07) is ~27 minutes; STRETCH (the real bug fix, HW-08..10) is ~8 minutes; CHALLENGE (written research, HW-11) is ~15 minutes.